

TRƯỜNG ĐẠI HỌC
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO TIỂU LUẬN
HỌC PHẦN: LƯU TRỮ DỮ LIỆU

GIẢI PHÁP LƯU TRỮ, SAO LƯU VÀ KHÔI PHỤC
THẨM HỌA ĐA SITE CHO HỆ THỐNG
DỰ BÁO THỜI TIẾT

(Triển khai trên MinIO Site Replication · PostgreSQL · MLflow · Docker)

Nhóm thực hiện: NHÓM 13 (LTDL-N13)

Thành viên:

.....
.....

Giảng viên hướng dẫn:

Hà Nội, tháng 10 năm 2026

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ.....	5
CHƯƠNG 1. GIỚI THIỆU	6
1.1. Bối cảnh và lý do chọn đề tài	6
1.2. Mục tiêu.....	6
1.3. Phạm vi và giới hạn.....	6
1.4. Phương pháp thực hiện.....	7
1.5. Cấu trúc báo cáo	7
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	8
2.1. Lưu trữ object và giao thức S3	8
2.2. Erasure coding và khả năng chịu lỗi mức drive	8
2.3. Versioning, Object Lock và Lifecycle.....	8
2.4. Site Replication	9
2.5. RPO, RTO và chiến lược sao lưu	9
2.6. Mã hóa phía máy chủ và quản lý khóa	9
2.7. MLflow và bài toán lưu trữ của hệ thống học máy	9
CHƯƠNG 3. KIẾN TRÚC ĐỀ XUẤT	11
3.1. Tổng quan.....	11
3.2. Các thành phần	11
3.3. Phân vùng mạng	12
3.4. Dung lượng khả dụng theo erasure coding.....	12
3.5. Luồng dữ liệu chính.....	13
3.6. Tự động hóa vận hành	13
CHƯƠNG 4. LOẠI DỮ LIỆU CẦN LƯU TRỮ.....	15
4.1. Nhận diện dữ liệu của hệ thống.....	15
4.2. Phân loại theo mức quan trọng và cơ chế bảo vệ.....	15
4.3. Lược đồ dữ liệu thô	16
CHƯƠNG 5. PHƯƠNG ÁN SAO LƯU VÀ KHÔI PHỤC ĐÁP ỨNG RPO	17
5.1. Chiến lược tổng thể: bốn lớp phòng thủ.....	17
5.2. Lớp 1 – Erasure coding	17
5.3. Lớp 2 – Versioning và ILM.....	17
5.4. Lớp 3 – Site Replication.....	18
5.5. Lớp 4 – Sao lưu PostgreSQL định kỳ vào bucket bất biến	18
5.5.1. Tiến trình backup.....	18
5.5.2. Object Lock COMPLIANCE	19
5.6. Bảng RPO/RTO cam kết và cơ sở kỹ thuật.....	19

CHƯƠNG 6. GIẢI PHÁP BẢO MẬT DỮ LIỆU.....	20
6.1. Quản lý bí mật	20
6.2. Phân quyền tối thiểu (least privilege)	20
6.3. Mã hóa phía máy chủ SSE-S3	21
6.4. Bất biến (WORM) và versioning	21
6.5. Audit log	21
6.6. Phân vùng mạng	22
6.7. Mã hóa đường truyền (TLS).....	22
CHƯƠNG 7. QUY TRÌNH KHÔI PHỤC DỮ LIỆU	23
7.1. Kịch bản 1 – Xóa nhầm hoặc ghi đè nhầm object.....	23
7.2. Kịch bản 2 – Hỏng hoặc mất dữ liệu trên drive	23
7.3. Kịch bản 3 – Khôi phục metadata MLflow (không mất site).....	24
7.4. Kịch bản 4 – Mất toàn bộ site A (failover sang site B)	24
Khôi phục site A (failback)	25
7.5. Kịch bản 5 – Ransomware / tài khoản quản trị bị lộ	25
7.6. Kiểm tra định kỳ	26
CHƯƠNG 8. TRIỂN KHAI VÀ KIỂM THỬ	27
8.1. Môi trường.....	27
8.2. Trình tự triển khai từ đầu.....	27
8.3. Kết quả huấn luyện (tải công việc thực tế).....	27
8.4. Kiểm thử các kịch bản sự cố	27
8.5. Sự cố gặp phải và bài học	28
CHƯƠNG 9. ĐÁNH GIÁ, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN.....	30
9.1. Đánh giá theo yêu cầu đề bài.....	30
9.2. Ưu điểm	30
9.3. Hạn chế	30
9.4. Hướng phát triển.....	31
KẾT LUẬN	32
TÀI LIỆU THAM KHẢO	33
PHỤ LỤC A. BẢN PHÂN CÔNG CÔNG VIỆC CỦA NHÓM	34
Nhật ký làm việc nhóm (tóm tắt).....	34
PHỤ LỤC B. MÃ NGUỒN VÀ CẤU HÌNH CHÍNH.....	36
B.1. Cấu trúc thư mục dự án.....	36
B.2. Trích docker-compose.yml – dịch vụ minio-a và backup-cron.....	36
B.3. Trích scripts/setup.sh – bucket, Object Lock, IAM.....	36
B.4. Policy db-backup-writer.json.....	37

B.5. scripts/restore_db.sh	37
B.6. Trích app/common.py – client S3 dùng user ứng dụng.....	37

(Mở file trong Word, nhấn chuột phải vào mục lục → Update Field → Update entire table để cập nhật số trang.)

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Từ viết tắt	Tiếng Anh	Giải nghĩa
DR	Disaster Recovery	Khôi phục thảm họa – khả năng khôi phục hệ thống sau sự cố lớn (mất site, cháy nổ, ransomware...)
RPO	Recovery Point Objective	Lượng dữ liệu tối đa chấp nhận mất, tính bằng thời gian (ví dụ RPO = 5 phút: mất tối đa 5 phút dữ liệu mới nhất)
RTO	Recovery Time Objective	Thời gian tối đa chấp nhận để đưa hệ thống hoạt động trở lại sau sự cố
EC	Erasure Coding	Mã hóa xóa – chia object thành K shard dữ liệu + M shard parity để chịu lỗi mất drive
S3	Simple Storage Service	Giao thức lưu trữ object do AWS đề xuất, được MinIO tương thích hoàn toàn
IAM	Identity and Access Management	Quản lý danh tính và phân quyền (user, policy)
ILM	Information Lifecycle Management	Quản lý vòng đời dữ liệu: tự động hết hạn, xóa version cũ
SSE-S3	Server-Side Encryption (S3 managed keys)	Mã hóa phía máy chủ, khóa do hệ thống lưu trữ quản lý
WORM	Write Once Read Many	Ghi một lần, đọc nhiều lần – dữ liệu không thể sửa/xóa trong thời gian retention
KMS	Key Management Service	Dịch vụ quản lý khóa mã hóa
TLS	Transport Layer Security	Mã hóa đường truyền (HTTPS)
JSONL	JSON Lines	Định dạng log: mỗi dòng một bản ghi JSON
MLflow	—	Nền tảng mã nguồn mở quản lý vòng đời mô hình học máy (tracking, registry, artifact)
mc	MinIO Client	Công cụ dòng lệnh quản trị MinIO

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh và lý do chọn đề tài

Trong các hệ thống dự báo thời tiết dựa trên học máy, dữ liệu là tài sản trung tâm: dữ liệu quan trắc lịch sử được tích lũy nhiều năm là nguồn duy nhất để huấn luyện lại mô hình; các mô hình đã huấn luyện (artifact) là kết quả trực tiếp phục vụ dự báo; siêu dữ liệu thí nghiệm (tham số, chỉ số đánh giá, phiên bản mô hình) là cơ sở để truy vết và tái lập kết quả. Mất bất kỳ lớp dữ liệu nào trong số này đều làm gián đoạn dịch vụ dự báo hoặc buộc phải huấn luyện lại từ đầu với chi phí lớn.

Thực tế vận hành cho thấy sự cố không chỉ đến từ hỏng hóc phần cứng đơn lẻ (hỏng ổ đĩa) mà còn từ thao tác nhầm của con người (xóa nhầm file), tấn công mã độc tổng tiền (mã hóa hoặc xóa bản sao lưu), và thảm họa quy mô site (mất điện kéo dài, cháy, mất kết nối trung tâm dữ liệu). Một giải pháp lưu trữ nghiêm túc phải xử lý được cả bốn nhóm rủi ro này với các mục tiêu RPO/RTO được định lượng rõ ràng.

Đề tài lựa chọn xây dựng và kiểm chứng một giải pháp lưu trữ – sao lưu – khôi phục thảm họa đa site cho hệ thống dự báo thời tiết Hà Nội, sử dụng hoàn toàn phần mềm mã nguồn mở (MinIO, PostgreSQL, MLflow) và đóng gói bằng Docker để có thể tái lập trên bất kỳ máy nào.

1.2. Mục tiêu

1. Đề xuất kiến trúc lưu trữ hai site (site chính A và site dự phòng B) với khả năng chịu lỗi ở cả mức drive (erasure coding) và mức site (site replication).
2. Phân loại các loại dữ liệu cần lưu trữ của hệ thống dự báo thời tiết theo mức độ quan trọng, từ đó gán cơ chế bảo vệ và mục tiêu RPO/RTO phù hợp cho từng loại.
3. Thiết kế phương án sao lưu và khôi phục đáp ứng $RPO \leq 5$ phút cho siêu dữ liệu và $RPO \approx$ vài giây cho dữ liệu object; $RTO \leq 30$ phút cho toàn hệ thống.
4. Xây dựng giải pháp bảo mật dữ liệu nhiều lớp: quản lý bí mật, phân quyền tối thiểu, mã hóa lưu trữ, bất biến (WORM), audit log, phân vùng mạng, TLS.
5. Xây dựng quy trình (runbook) khôi phục dữ liệu cho từng kịch bản sự cố và kiểm chứng bằng thực nghiệm, đo thời gian thực tế.

1.3. Phạm vi và giới hạn

- Hệ thống được triển khai dạng mô phỏng trên một máy tính cá nhân bằng Docker Compose: hai site là hai nhóm container trên hai mạng Docker tách biệt; bốn drive của mỗi MinIO là bốn Docker volume. Điều này đủ để kiểm chứng logic chịu lỗi, replication và quy trình khôi phục, nhưng không mô phỏng được độ trễ mạng WAN thật giữa hai trung tâm dữ liệu.

- Dữ liệu sử dụng là dữ liệu thời tiết Hà Nội theo ngày từ 2016 đến 2025 (nguồn Open-Meteo Archive, 3 653 ngày); mô hình dự báo là RandomForest dự đoán nhiệt độ trung bình ngày kế tiếp. Chất lượng mô hình không phải trọng tâm của đề tài mà chỉ là tải công việc thực tế để sinh ra dữ liệu cần bảo vệ.
- TLS được thiết kế và chuẩn bị sẵn (script sinh chứng chỉ, file compose override) nhưng tắt mặc định để đơn giản hóa demo; báo cáo trình bày cách bật.

1.4. Phương pháp thực hiện

Nhóm thực hiện theo tiến trình: (1) khảo sát yêu cầu và phân loại dữ liệu; (2) nghiên cứu các cơ chế của MinIO, PostgreSQL, MLflow liên quan đến lưu trữ và DR; (3) thiết kế kiến trúc và bảng RPO/RTO; (4) triển khai bằng Docker Compose và script tự động hóa (Makefile, bash); (5) kiểm thử từng kịch bản sự cố, ghi nhận số liệu; (6) đánh giá, chỉ ra hạn chế và hướng phát triển. Toàn bộ mã nguồn, cấu hình và tài liệu vận hành được tổ chức trong một thư mục dự án duy nhất để có thể tái lập bằng một chuỗi lệnh.

1.5. Cấu trúc báo cáo

Báo cáo gồm 9 chương: Chương 2 trình bày cơ sở lý thuyết; Chương 3 mô tả kiến trúc đề xuất; Chương 4 phân loại dữ liệu; Chương 5 trình bày phương án sao lưu – khôi phục và bảng RPO/RTO; Chương 6 trình bày giải pháp bảo mật; Chương 7 là các quy trình khôi phục dữ liệu chi tiết; Chương 8 mô tả triển khai và kết quả kiểm thử; Chương 9 đánh giá và hướng phát triển. Phần phụ lục gồm bản phân công công việc và các cấu hình chính.

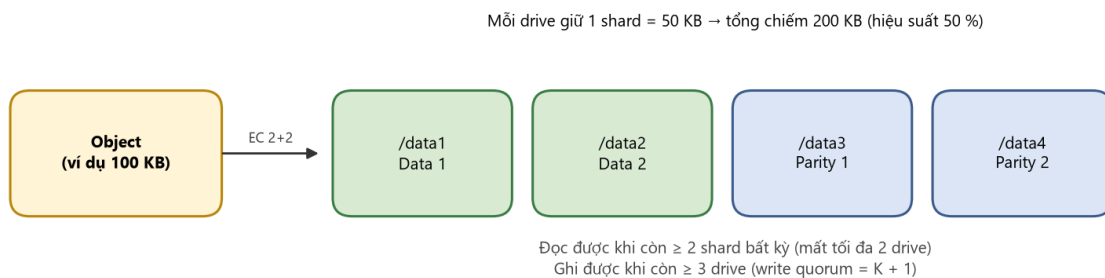
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Lưu trữ object và giao thức S3

Lưu trữ object (object storage) quản lý dữ liệu dưới dạng các object phẳng trong bucket, mỗi object gồm nội dung, siêu dữ liệu và một khóa (key) duy nhất, thay vì cây thư mục như file system hay block như SAN. Mô hình này mở rộng theo chiều ngang tốt, phù hợp với dữ liệu phi cấu trúc dung lượng lớn như file CSV quan trắc, artifact mô hình, log. Giao thức S3 của Amazon đã trở thành chuẩn de facto; MinIO là hệ thống lưu trữ object mã nguồn mở tương thích S3, có thể triển khai on-premise, hỗ trợ đầy đủ versioning, object lock, mã hóa, lifecycle và replication.

2.2. Erasure coding và khả năng chịu lỗi mức drive

Thay vì nhân bản toàn bộ object (replication $\times 3$ tốn 200 % dung lượng dư thừa), erasure coding chia object thành K shard dữ liệu và tính thêm M shard parity bằng mã Reed-Solomon; $N = K + M$ shard được rải trên N drive khác nhau. Object khôi phục được từ bất kỳ K shard nào, tức chịu được mất tối đa M drive. Hiệu suất dung lượng là K/N .



Hình 2.1. Erasure coding EC 2+2 trên 4 drive: mất 2 drive bất kỳ vẫn đọc được

MinIO tự chọn parity theo số drive trong một erasure set: với 4 drive, mặc định EC:2 (2 data + 2 parity). Hai ngưỡng quan trọng cần phân biệt: đọc được khi còn $\geq K$ drive; ghi được khi còn $\geq K + 1$ drive (write quorum) để tránh tình trạng split-brain khi $K = M$. Với cấu hình 4 drive, hệ thống đọc được khi mất 2 drive nhưng chỉ ghi được khi mất tối đa 1 drive.

2.3. Versioning, Object Lock và Lifecycle

- **Versioning:** mỗi lần ghi đè cùng key tạo một version mới thay vì thay thế; xóa chỉ đặt một delete marker. Nhờ đó xóa nhầm hay ghi đè nhầm đều khôi phục được. Versioning là điều kiện bắt buộc để bật replication.
- **Object Lock (WORM):** gán thời hạn retention cho từng version. Chế độ GOVERNANCE cho phép user có quyền đặc biệt bỏ qua; chế độ COMPLIANCE

thì không ai – kể cả root – xóa hay rút ngắn retention được cho đến khi hết hạn. Đây là lớp bảo vệ chính chống ransomware và xóa nhầm bản sao lưu.

- **Lifecycle (ILM):** quy tắc tự động theo thời gian: hết hạn version hiện tại (Expiration), xóa version cũ (NoncurrentVersionExpiration), dọn delete marker mờ côi. ILM giúp kiểm soát dung lượng tăng do versioning và backup định kỳ.

2.4. Site Replication

MinIO Site Replication liên kết nhiều cụm MinIO độc lập thành một nhóm active-active: mọi bucket, object, version, delete marker, cấu hình bucket (versioning, lock, SSE, ILM), user, policy, service account tạo ở bất kỳ site nào đều được tự động nhân bản sang các site còn lại. Replication là bất đồng bộ – ghi trả về thành công ngay ở site nguồn rồi mới đẩy sang site đích – nên RPO khác 0 nhưng thường chỉ vài giây trong mạng nội bộ. Ưu điểm so với bucket replication đơn lẻ là IAM cũng được đồng bộ, nên khi failover ứng dụng dùng lại đúng credential cũ.

2.5. RPO, RTO và chiến lược sao lưu

RPO (Recovery Point Objective) là khoảng thời gian dữ liệu tối đa chấp nhận mất; RTO (Recovery Time Objective) là thời gian tối đa để dịch vụ hoạt động trở lại. Hai chỉ số này quyết định lựa chọn kỹ thuật: RPO càng nhỏ đòi hỏi replication liên tục hoặc chu kỳ backup càng ngắn; RTO càng nhỏ đòi hỏi hạ tầng dự phòng sẵn sàng (warm/hot standby) và quy trình tự động hóa. Nguyên tắc 3-2-1 (3 bản sao, 2 phương tiện, 1 bản ở nơi khác) và biến thể 3-2-1-1-0 (thêm 1 bản bất biến, 0 lỗi khi kiểm tra) là kim chỉ nam khi thiết kế.

Với cơ sở dữ liệu quan hệ, hai hướng phổ biến là logical backup (pg_dump: nhất quán, độc lập phiên bản, dễ khôi phục chọn lọc, nhưng RPO bằng chu kỳ chạy) và physical/streaming replication (RPO gần 0 nhưng phức tạp hơn và không chống được xóa nhầm vì lệnh DELETE cũng được nhân bản). Đề tài chọn pg_dump định kỳ kết hợp lưu bản dump vào bucket bất biến, đáp ứng RPO 5 phút và đồng thời chống ransomware.

2.6. Mã hóa phía máy chủ và quản lý khóa

SSE-S3 mã hóa từng object bằng khóa dữ liệu (DEK) riêng, DEK lại được mã hóa bằng khóa gốc (KEK) do KMS quản lý; dữ liệu trên đĩa luôn là ciphertext, chỉ giải mã khi có request hợp lệ qua API. MinIO hỗ trợ KMS tĩnh một khóa (biến môi trường, cho môi trường thử nghiệm) hoặc KES kết nối HashiCorp Vault / AWS KMS / HSM cho production. Khi replicate object SSE-S3 giữa hai site, hai site phải giải mã được cùng khóa, nghĩa là dùng chung KMS hoặc cùng khóa gốc.

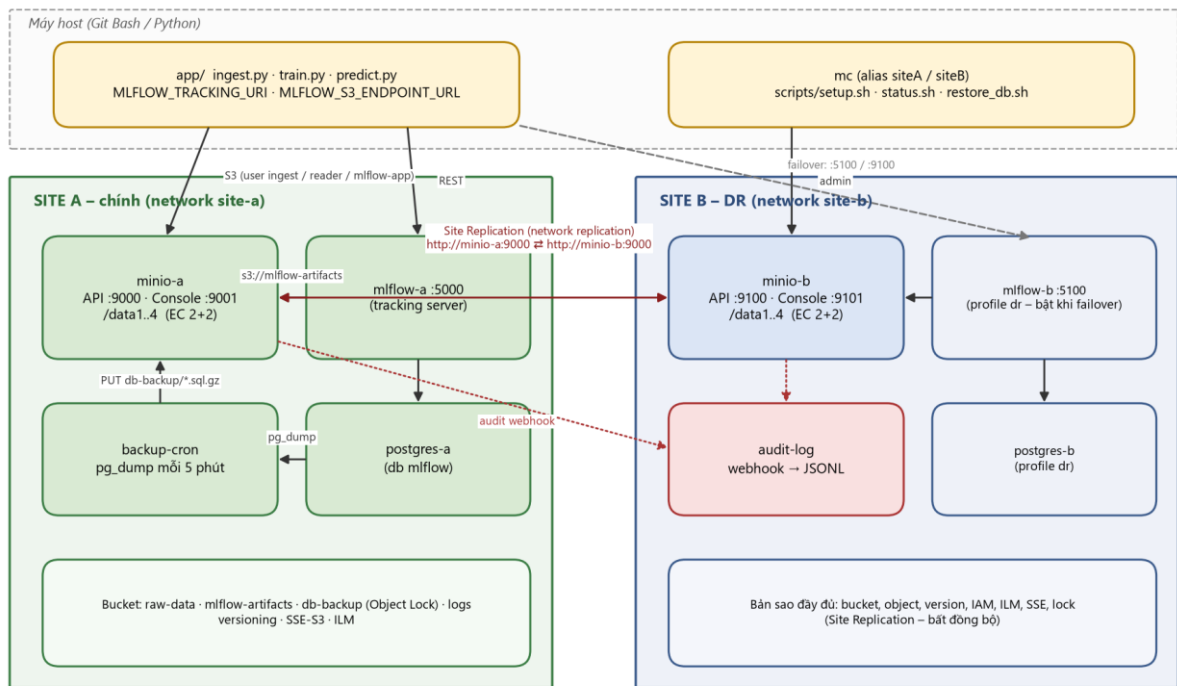
2.7. MLflow và bài toán lưu trữ của hệ thống học máy

MLflow tách dữ liệu thành hai kho: backend store (cơ sở dữ liệu quan hệ – PostgreSQL) chứa run, param, metric, model registry; và artifact store (object storage – S3/MinIO) chứa file model, input example, requirement. Hai kho này phải nhất quán với nhau: registry trỏ tới đường dẫn artifact trên S3. Vì vậy phương án DR phải bảo vệ đồng thời cả hai và quy trình khôi phục phải đưa cả hai về cùng một thời điểm.

CHƯƠNG 3. KIẾN TRÚC ĐỀ XUẤT

3.1. Tổng quan

Kiến trúc gồm hai site đối xứng. Site A là site chính, chạy toàn bộ dịch vụ: MinIO (lưu trữ object), PostgreSQL (backend store MLflow), MLflow tracking server và tiến trình sao lưu định kỳ. Site B là site dự phòng (DR): MinIO luôn chạy để nhận replication liên tục; PostgreSQL và MLflow ở trạng thái warm standby – được định nghĩa sẵn nhưng chỉ khởi động khi failover. Một dịch vụ audit-log dùng chung nhận webhook từ cả hai MinIO.



Hình 3.1. Kiến trúc hai site và luồng dữ liệu

3.2. Các thành phần

Thành phần	Image / phiên bản	Vai trò	Cổng host
minio-a	minio RELEASE.2025-04-22	Lưu trữ object site A, 4 drive EC:2, KMS tĩnh, audit webhook	9000 (API), 9001 (Console)
postgres-a	postgres:16.15-alpine	Backend store MLflow (db mlflow)	không publish
mlflow-a	mlflow v3.16.1 + psycopg2 + boto3	Tracking server, artifact root s3://mlflow-artifacts, chế độ --no-serve-artifacts (client ghi thẳng S3)	5000
backup-cron	postgres:16.15-alpine + mc	crond chạy pg_dump mỗi 5 phút → gzip → PUT lên db-backup bằng user riêng	—

Thành phần	Image / phiên bản	Vai trò	Cổng host
minio-b	minio RELEASE.2025-04-22	Lưu trữ object site B, nhận Site Replication	9100, 9101
postgres-b	postgres:16.15-alpine	Backend store dự phòng (profile dr)	không publish
mlflow-b	mlflow v3.16.1	Tracking server dự phòng (profile dr)	5100
audit-log	python:3.12-alpine	Nhận audit webhook, ghi JSONL vào volume audit-logs	—
mc (profile tools)	minio/mc	Chạy trong network replication để cấu hình Site Replication	—

Mỗi container có giới hạn bộ nhớ (mem_limit) và healthcheck riêng; các dịch vụ phụ thuộc chỉ khởi động khi dịch vụ nền đã healthy (depends_on: condition: service_healthy), nhờ đó lệnh docker compose up --wait trả về khi toàn bộ hệ thống thực sự sẵn sàng.

3.3. Phân vùng mạng

Network	Thành viên	Mục đích
site-a	minio-a, postgres-a, mlflow-a, backup-cron, audit-log	Mạng nội bộ site A; PostgreSQL không publish cổng ra host
site-b	minio-b, postgres-b, mlflow-b, audit-log	Mạng nội bộ site B
replication	minio-a, minio-b (+ mc khi setup)	Kênh duy nhất hai site nhìn thấy nhau, mô phỏng đường WAN giữa hai trung tâm dữ liệu

Endpoint ngang hàng của Site Replication là http://minio-a:9000 và http://minio-b:9000 – tên nội bộ trên network replication. Không thể dùng localhost:9100 vì bên trong container minio-a, localhost là chính nó. Đây là lỗi thiết kế thường gặp khi mô phỏng đa site trên một máy và được xử lý bằng cách chạy mc trong một container thuộc network replication khi thiết lập.

3.4. Dung lượng khả dụng theo erasure coding

Mỗi site có 1 node $\times$ 4 drive nên một erasure set có stripe = 4; MinIO chọn parity mặc định EC:2 (lệnh mc admin info báo 4 drives online, EC:2). Bảng dưới so sánh các lựa chọn:

Storage class	Data (K) + Parity (M)	Hiệu suất K/N	Hỏng tối đa vẫn ĐỌC	Hỏng tối đa vẫn GHI	4 $\times$ 100 GiB
STANDARD (đang dùng)	2 + 2	50 %	2 drive	1 drive (quorum K+1 = 3)	200 GiB

Storage class	Data (K) + Parity (M)	Hiệu suất K/N	Hồng tối đa vẫn ĐỌC	Hồng tối đa vẫn GHI	4 × 100 GiB
REDUCED_REDUNDANCY	3 + 1	75 %	1 drive	1 drive	300 GiB
EC:1 (nếu cấu hình)	3 + 1	75 %	1 drive	1 drive	300 GiB

Vì hai site nhân bản đầy đủ (active-active), dung lượng khả dụng cho dữ liệu người dùng là $\min(\text{site A}, \text{site B}) = 50\%$ dung lượng thô của một site, tức 25 % tổng dung lượng thô của cả hai site. Đổi lại, hệ thống chịu được mất 2 drive trên mỗi site hoặc mất nguyên một site. Đây là đánh đổi có chủ đích: với dữ liệu quan trọng như dữ liệu lịch sử và mô hình, độ bền được ưu tiên hơn hiệu suất dung lượng.

3.5. Luồng dữ liệu chính

1. **Ingest:** ingest.py tải dữ liệu Open-Meteo (hoặc sinh dữ liệu giả lập có tính mùa vụ nếu mất mạng), chia theo tháng và PUT từng lô lên raw-data/weather/hanoi/YYYY/YYYY-MM.csv bằng user ingest-writer (chỉ có quyền ghi). Chạy lại ghi đè cùng key → tạo version mới.
2. **Train:** train.py đọc toàn bộ lô bằng user raw-reader (chỉ đọc), xây dựng đặc trưng (lag 1–3 ngày, trung bình trượt 7 ngày, sin/cos ngày trong năm), huấn luyện RandomForest, log param/metric vào MLflow (PostgreSQL) và ghi artifact model thẳng lên s3://mlflow-artifacts bằng user mlflow-app.
3. **Predict:** predict.py lấy version mới nhất từ Model Registry, tải artifact từ MinIO, dự báo nhiệt độ ngày mai.
4. **Backup:** backup-cron chạy pg_dump --clean --if-exists mỗi 5 phút, nén gzip, kiểm tra toàn vẹn (gzip -t), PUT lên db-backup bằng user db-backup-writer.
5. **Replication:** mọi object, version, delete marker, cấu hình và IAM trên site A tự động sang site B (và ngược lại).
6. **Failover:** ứng dụng chỉ cần đổi hai biến môi trường (MLFLOW_TRACKING_URI → :5100, MLFLOW_S3_ENDPOINT_URL → :9100) để làm việc với site B.

3.6. Tự động hóa vận hành

Toàn bộ thao tác được gom vào Makefile để mỗi thành viên và người chấm chạy được cùng một kết quả:

Lệnh	Thực hiện
make env	Sinh .env với toàn bộ mật khẩu/ khóa ngẫu nhiên (không ghi đè nếu đã có)
make up	Build image và khởi động site A, minio-b, backup-cron, audit-log; chờ healthy

Lệnh	Thực hiện
make setup	Alias, Site Replication, bucket, versioning, Object Lock, SSE-S3, ILM, IAM user/policy, kiểm tra đồng bộ (idempotent)
make ingest / train / predict	Chạy ba bước ML trong virtualenv Python
make backup	Chạy backup DB ngay lập tức, ngoài lịch 5 phút
make status	Trạng thái replication, số object từng bucket hai site, 5 bản backup mới nhất
make dr-up	Bật postgres-b + mlflow-b (profile dr)
make dr-restore	Nạp bản dump mới nhất từ siteB/db-backup vào postgres-b, khởi động lại mlflow-b
make certs	Sinh CA nội bộ và chứng chỉ cho minio-a/minio-b (TLS)
make down / make clean	Dừng (giữ dữ liệu) / dừng và xóa toàn bộ volume

CHƯƠNG 4. LOẠI DỮ LIỆU CẦN LƯU TRỮ

4.1. Nhận diện dữ liệu của hệ thống

Hệ thống dự báo thời tiết dựa trên học máy sinh ra sáu nhóm dữ liệu có đặc tính và mức quan trọng khác nhau. Việc phân loại là bước đầu tiên và quyết định: cùng một cơ chế bảo vệ áp cho mọi dữ liệu sẽ hoặc quá tốn kém (bảo vệ log như bảo vệ mô hình) hoặc quá rủi ro (bảo vệ backup như bảo vệ log).

Nhóm dữ liệu	Dạng / vị trí	Kích thước & tốc độ tăng	Đặc tính
Dữ liệu thời tiết thô	CSV theo tháng, bucket raw-data, prefix weather/hanoi/YYYY/	120 lô $\approx$ 3 653 dòng; tăng 1 lô/tháng; ghi đề khi cập nhật	Nguồn để huấn luyện lại; không tái tạo được nếu nguồn ngoài ngừng cung cấp
Artifact mô hình	model.pkl, MLmodel, requirements...; bucket mlflow-artifacts	$\approx$ 7 object/run, vài MB	Phục vụ dự báo trực tiếp; tái tạo được nhưng tốn thời gian train
Metadata MLflow	Bảng run, param, metric, registered_model, model_version; PostgreSQL	$\approx$ 12 KB/dump nén; tăng theo số run	Liên kết registry $\leftrightarrow$ artifact; mất là mất khả năng truy vết
Bản backup DB	mlflow_YYYYmmdd_HHMMSS.sql.gz; bucket db-backup	$\approx$ 12 KB $\times$ 288 bản/ngày	Nguồn khôi phục cuối cùng; phải bất biến
Log ứng dụng	bucket logs	nhỏ, hết hạn 30 ngày	Giá trị giảm nhanh theo thời gian
Audit log MinIO	audit-site-a.jsonl, audit-site-b.jsonl; volume audit-logs	1 dòng/request S3	Phục vụ điều tra sự cố, không phục vụ vận hành

4.2. Phân loại theo mức quan trọng và cơ chế bảo vệ

Dữ liệu	Mức quan trọng	Cơ chế bảo vệ	RPO mục tiêu	RTO mục tiêu
Dữ liệu thô (raw-data)	Cao	EC:2 · versioning (noncurrent giữ 14 ngày) · Site Replication · SSE-S3	$\approx$ vài giây	$\leq$ 15 phút (đổi endpoint sang :9100)
Artifact model (mlflow-artifacts)	Rất cao	EC:2 · versioning · Site Replication · SSE-S3	$\approx$ vài giây	$\leq$ 30 phút (bật mlflow-b)
Metadata MLflow (postgres-a)	Rất cao	pg_dump 5 phút/lần $\rightarrow$ db-backup (được replicate)	$\leq$ 5 phút	$\leq$ 30 phút (dr-up + dr-restore)
Bản backup DB (db-backup)	Tối quan trọng	Object Lock COMPLIANCE 1 ngày (WORM) · versioning · Site Replication · SSE-S3 · ILM 7 ngày · user chỉ ghi	= RPO của DB	— (là nguồn khôi phục)
Log ứng dụng (logs)	Thấp	Site Replication · ILM hết hạn 30 ngày	Best effort	$\leq$ 24 giờ
Audit log	Trung bình	Webhook $\rightarrow$ JSONL trên volume riêng	Best effort	Không yêu cầu

Ba nguyên tắc rút ra từ bảng: (1) dữ liệu càng quan trọng càng nhiều lớp bảo vệ độc lập (drive → site → thời gian → bất biến); (2) bản sao lưu phải được bảo vệ mạnh hơn dữ liệu gốc vì nó là tuyến phòng thủ cuối; (3) dữ liệu giá trị thấp phải có cơ chế tự hết hạn để không tiêu tốn dung lượng và làm chậm replication.

4.3. Lược đồ dữ liệu thô

Mỗi lô CSV có sáu cột: date, temp_mean, temp_max, temp_min, precipitation, wind_max; kèm metadata object source (open-meteo-archive hoặc synthetic-seasonal) và rows. Việc chia theo tháng với key cố định YYYY/YYYY-MM.csv có hai lợi ích: ingest lại một tháng chỉ ghi đè một object (tạo version mới, minh họa versioning + ILM), và load_raw() gộp các lô rồi drop_duplicates("date", keep="last") nên dữ liệu cập nhật luôn thắng.

CHƯƠNG 5. PHƯƠNG ÁN SAO LƯU VÀ KHÔI PHỤC ĐÁP ỨNG RPO

5.1. Chiến lược tổng thể: bốn lớp phòng thủ

Phương án được thiết kế theo bốn lớp, mỗi lớp chống một nhóm rủi ro khác nhau và độc lập với các lớp còn lại:

Lớp	Cơ chế	Chống rủi ro	Phạm vi
1. Drive	Erasur coding EC 2+2 trong mỗi MinIO	Hỏng/xóa nội dung tối đa 2 drive	Mọi object
2. Thời gian	Versioning + ILM (noncurrent 14 ngày, logs 30 ngày)	Xóa nhầm, ghi đè nhầm	Mọi bucket
3. Site	Site Replication bất đồng bộ A ⇌ B	Mất nguyên site (điện, mạng, cháy)	Object + cấu hình + IAM
4. Bất biến	pg_dump 5 phút → db-backup với Object Lock COMPLIANCE	Ransomware, xóa bản sao lưu, root bị lộ	Metadata MLflow

Đối chiếu với nguyên tắc 3-2-1-1-0: mỗi object có ≥ 2 bản (site A, site B) cộng các version cũ; hai site là hai hệ thống lưu trữ độc lập; site B đóng vai trò bản sao ở nơi khác; db-backup là bản bất biến; gzip -t và kiểm tra đồng bộ trong setup.sh là bước xác nhận 0 lỗi.

5.2. Lớp 1 – Erasure coding

Cấu hình server /data{1...4} khiến MinIO tạo một erasure set 4 drive với EC:2. Không cần cấu hình thêm; mọi object PUT vào đều tự động được chia thành 2 shard dữ liệu + 2 shard parity. Object nhỏ (dưới 128 KiB) được lưu inline trong file xl.meta trên từng drive. Khi một drive mất dữ liệu, thao tác đọc vẫn thành công từ các shard còn lại và MinIO tự heal shard thiếu; quản trị viên có thể chủ động quét bằng mc admin heal -r.

5.3. Lớp 2 – Versioning và ILM

Versioning được bật cho cả bốn bucket (bắt buộc với Site Replication). ILM được nhập bằng mc ilm import (thay toàn bộ cấu hình, nên chạy lại không sinh rule trùng):

```
# logs: hết hạn sau 30 ngày
{"Rules":[{"ID":"logs-expire-30d","Status":"Enabled","Filter":{"Prefix":""},"Expiration":{"Days":30}}]}
# raw-data: version cũ hết hạn sau 14 ngày
{"Rules":[{"ID":"raw-noncurrent-14d","Status":"Enabled","Filter":{"Prefix":"","NoncurrentVersionExpiration":{"NoncurrentDays":14}}]}
# db-backup: xóa bản backup sau 7 ngày + dọn delete marker
{"Rules":[{"ID":"backup-expire-7d","Status":"Enabled","Filter":{"Prefix":"","Expiration":{"Days":7},"NoncurrentVersionExpiration":{"NoncurrentDays":1}},{"ID":"backup-clean-delmarker","Status":"Enabled","Filter":{"Prefix":"","Expiration":{"ExpiredObjectDeleteMarker":true}}]}]}
```

Điểm cần lưu ý với bucket có versioning: Expiration chỉ đặt delete marker, dữ liệu vẫn còn dưới dạng noncurrent version; vì vậy rule cho db-backup phải kết hợp NoncurrentVersionExpiration và ExpiredObjectDeleteMarker để dữ liệu thực sự được giải phóng sau ≈ 8 ngày. Với chu kỳ 5 phút, số bản backup giữ lại ổn định ở khoảng $2\,000 - 2\,300 \text{ bản} \times 12 \text{ KB} \approx 27 \text{ MB}$ – hoàn toàn chấp nhận được.

5.4. Lớp 3 – Site Replication

Site Replication được thiết lập trước khi tạo bất kỳ bucket nào (site B phải trống khi tham gia). Lệnh cấu hình chạy trong container mc thuộc network replication:

```
compose run --rm --no-deps \
-e "MC_HOST_a=http://${MINIO_A_ROOT_USER}:${MINIO_A_ROOT_PASSWORD}@minio-a:9000" \
-e "MC_HOST_b=http://${MINIO_B_ROOT_USER}:${MINIO_B_ROOT_PASSWORD}@minio-b:9000" \
mc --no-color admin replicate add a b --replicate-ilm-expiry
```

Sau đó mọi thao tác cấu hình (bucket, versioning, lock, SSE, ILM, user, policy) chỉ cần làm trên site A. Script setup.sh có vòng chờ tối đa 60 giây để xác nhận bucket, user và ILM đã xuất hiện ở site B trước khi báo hoàn tất. Trạng thái được xem bằng mc admin replicate status: Buckets 4/4, Policies 9/9, Users 4/4, ILM Expiry Rules 3/3 in sync; hàng đợi object Queued = 0.

Một phát hiện thực nghiệm: Site Replication không nhân bản rule ExpiredObjectDeleteMarker, nên setup.sh nhập rule ILM của db-backup lên cả hai site thay vì chỉ site A.

5.5. Lớp 4 – Sao lưu PostgreSQL định kỳ vào bucket bất biến

5.5.1. Tiến trình backup

Container backup-cron được build từ image postgres:16.15-alpine (cùng major version với postgres-a để pg_dump tương thích) và bổ sung binary mc. Entrypoint lưu biến môi trường ra /etc/backup.env (busybox crond không truyền env cho job), ghi crontab */5 * * * * và chạy crond ở foreground. Script backup_db.sh:

```
name="mlflow_$(date +%Y%m%d_%H%M%S).sql.gz"
pg_dump --no-owner --clean --if-exists | gzip -9 > "$tmp/$name"
gzip -t "$tmp/$name" # kiểm tra toàn vẹn trước khi đẩy
mc --quiet cp "$tmp/$name" "siteA/db-backup/$name"
mc stat "siteA/db-backup/$name" | grep -i encrypt # xác nhận SSE-S3
```

- --clean --if-exists: dump chứa lệnh DROP trước CREATE, nên khi restore vào database đã có dữ liệu cũ vẫn ra trạng thái đúng.
- Dùng user db-backup-writer với policy chỉ có ListBucket, GetObject, PutObject – không có DeleteObject. Kể cả khi credential này bị lộ, kẻ tấn công không xóa được bản sao lưu.

- Bản dump PUT lên site A rồi được replicate sang site B; khi failover, restore_db.sh đọc từ siteB/db-backup.

5.5.2. Object Lock COMPLIANCE

Bucket db-backup được tạo bằng mc mb --with-lock và đặt retention mặc định mc retention set --default COMPLIANCE 1d. Mọi object PUT vào tự động nhận Retain-Until-Date = thời điểm ghi + 1 ngày. Trong thời gian này không thể xóa version, kể cả bằng tài khoản root; lệnh mc rm --version-id bị từ chối. Thời hạn 1 ngày là đủ để phát hiện và xử lý sự cố ransomware trong khi vẫn cho phép ILM 7 ngày dọn dẹp bình thường (đến ngày thứ 7 các version đã hết retention).

5.6. Bảng RPO/RTO cam kết và cơ sở kỹ thuật

Dữ liệu	RPO mục tiêu	Cơ sở kỹ thuật	RTO mục tiêu	Thao tác khôi phục
raw-data, mlflow- artifacts, logs	≈ vài giây	Site Replication bất đồng bộ; đo bằng mc admin replicate status (Queued)	≤ 15 phút	Đổi MLFLOW_S3_ENDPOINT_URL sang :9100
Metadata MLflow	≤ 5 phút	Chu kỳ cron BACKUP_SCHEDULE=*/5; đổi được trong compose	≤ 30 phút	make dr-up && make dr-restore
Toàn hệ thống (mất site A)	≤ 5 phút	Bị chi phối bởi RPO của metadata	≤ 30 phút	Runbook 7.4; đo thực tế 45 giây
Xóa nhầm object	0	Versioning giữ mọi version	≤ 5 phút	mc undo / mc cp --version-id
Hỏng ≤ 2 drive một site	0	EC 2+2	0 (đọc tiếp) / vài phút (heal)	mc admin heal -r

Nếu yêu cầu RPO gần 0 cho metadata, giải pháp mở rộng là streaming replication PostgreSQL A → B (ngoài phạm vi demo) và giữ nguyên pg_dump định kỳ như lớp chống xóa nhầm – vì replication ở mức DB cũng nhận bản cả lệnh DELETE.

CHƯƠNG 6. GIẢI PHÁP BẢO MẬT DỮ LIỆU

Bảo mật được thiết kế theo nguyên tắc phòng thủ nhiều lớp (defense in depth): mỗi lớp giả định lớp trước có thể thất bại. Bảng tổng quan:

#	Lớp	Cách làm	Trạng thái
1	Quản lý bí mật	Mọi mật khẩu/ khóa trong .env sinh ngẫu nhiên, đã .gitignore; compose chỉ tham chiếu \${VAR}	Bật
2	Least privilege (IAM)	4 user ứng dụng với policy tối thiểu; root chỉ dùng cho setup	Bật
3	Mã hóa lưu trữ (SSE-S3)	KMS tĩnh + mc encrypt set sse-s3 cho cả 4 bucket	Bật
4	Bất biến (WORM)	db-backup: Object Lock COMPLIANCE 1 ngày	Bật
5	Versioning	Cả 4 bucket	Bật
6	Audit log	Webhook → container audit-log → JSONL	Bật
7	Phân vùng mạng	3 network Docker; Postgres không publish cổng	Bật
8	Mã hóa đường truyền (TLS)	gen_certs.sh + docker-compose.tls.yml	Chuẩn bị sẵn, tắt mặc định

6.1. Quản lý bí mật

File .env.example là mẫu với placeholder __RANDOM__ và __KMS_KEY__; scripts/gen_env.sh thay chúng bằng chuỗi ngẫu nhiên 24 ký tự [A-Za-z0-9] (đọc từ /dev/urandom) và khóa 32 byte base64. Chỉ dùng ký tự chữ-số vì mật khẩu được nhúng vào URL (postgresql://..., MC_HOST_*). File .env được liệt kê trong .gitignore; docker-compose.yml không chứa bất kỳ giá trị bí mật nào (kiểm chứng: grep -nE 'PASSWORD|SECRET' docker-compose.yml chỉ thấy \${...}). Hai site dùng root user khác nhau (admin-a, admin-b) với mật khẩu khác nhau.

6.2. Phân quyền tối thiểu (least privilege)

Không thành phần ứng dụng nào dùng tài khoản root. Bốn user được tạo bằng mc admin user add và gán policy JSON riêng:

User	Policy	Được phép	Bị cấm	Dùng bởi
mlflow-app	mlflow-artifacts-rw	List/Get/Put/Delete trong mlflow-artifacts	Mọi bucket khác	mlflow-a/b, train.py, predict.py
raw-reader	raw-data-readonly	List/Get raw-data	Ghi, xóa	train.py, predict.py
ingest-writer	raw-data-ingest	List/Put raw-data	Xóa, đọc bucket khác	ingest.py
db-backup-writer	db-backup-writer	List/Get/Put db-backup	Xóa (kết hợp Object Lock)	backup-cron, restore_db.sh

Ví dụ policy raw-data-ingest chỉ cho phép s3:PutObject trên arn:aws:s3:::raw-data/* và s3:ListBucket/GetBucketLocation trên bucket – tiến trình ingest không thể đọc dữ liệu đã có và không thể xóa. Site Replication tự đồng bộ user và policy sang site B, nên khi failover ứng dụng dùng lại đúng credential. Kết quả kiểm chứng thực nghiệm (Chương 8): mlflow-app ghi vào raw-data → Insufficient permissions; raw-reader ghi → Insufficient permissions; mlflow-app gọi mc admin info → Access Denied.

6.3. Mã hóa phía máy chủ SSE-S3

MinIO bản community không đi kèm KES; đề tài dùng KMS tĩnh một khóa MINIO_KMS_SECRET_KEY=minio-kms:<32 byte base64> – cách MinIO hỗ trợ cho môi trường thử nghiệm. Hai site dùng cùng một khóa để object SSE-S3 nhân bản sang B vẫn giải mã được. Auto-encryption được bật cho cả bốn bucket bằng mc encrypt set sse-s3, nên client không cần gửi header mã hóa. Kiểm chứng: đọc trực tiếp volume Docker của minio-a và grep chuỗi temp_mean (header CSV) trả về 0 file – dữ liệu trên đĩa là ciphertext; đọc qua API bằng user có quyền vẫn ra plaintext.

Hạn chế: khóa tĩnh nằm trong biến môi trường, không có xoay khóa hay HSM; lộ .env là lộ khóa. Production cần KES kết nối Vault/AWS KMS/HSM, thay MINIO_KMS_SECRET_KEY bằng MINIO_KMS_KES_ENDPOINT, MINIO_KMS_KES_KEY_NAME.

6.4. Bất biến (WORM) và versioning

Đã trình bày ở mục 5.3 và 5.5.2. Về mặt bảo mật, hai cơ chế này biến kịch bản tấn công phổ biến nhất – chiếm được credential rồi xóa/mã hóa cả dữ liệu lẫn backup – thành không khả thi: dữ liệu bị ghi đè vẫn còn version cũ; bản backup không xóa được trong 24 giờ dù có quyền root. Lưu ý vận hành: COMPLIANCE không cho ai rút ngắn retention, nên make clean phải xóa hẳn volume Docker chứ không thể xóa object.

6.5. Audit log

Cả hai MinIO được cấu hình MINIO_AUDIT_WEBHOOK_ENABLE_audit=on trở tới http://audit-log:8080/site-a hoặc /site-b. Container audit-log (Python, 30 dòng, giới hạn 64 MB RAM) ghi mỗi sự kiện một dòng JSON vào /logs/audit-<site>.jsonl và in tóm tắt: thời gian, accessKey, tên API, bucket/object, mã HTTP. Đường dẫn POST được kiểm tra bằng biểu thức chính quy site-[a-z] để chống path traversal ghi ra ngoài thư mục /logs. Mọi lần truy cập bị từ chối ở mục 6.2 hiện ra với statusCode 403 kèm accessKey – cơ sở để phát hiện dò quét quyền. Production nên đẩy webhook vào SIEM (Elasticsearch/Loki/Splunk).

6.6. Phân vùng mạng

Ba network Docker (site-a, site-b, replication) đảm bảo: dịch vụ site A không gọi trực tiếp được PostgreSQL site B và ngược lại; chỉ hai MinIO nói chuyện với nhau qua network replication; PostgreSQL hai site hoàn toàn không publish cổng ra host, nên chỉ truy cập được từ bên trong network tương ứng. Kiểm chứng bằng docker network inspect ltdl-dr_replication chỉ thấy minio-a và minio-b.

6.7. Mã hóa đường truyền (TLS)

TLS tắt mặc định vì yêu cầu demo dùng mc với http://localhost và chúng chỉ self-signed làm demo phức tạp thêm; tuy nhiên toàn bộ thành phần đã chuẩn bị sẵn:

- scripts/gen_certs.sh: sinh CA nội bộ (CN = LTDL-N13 Demo CA) và chứng chỉ cho minio-a, minio-b với SAN gồm tên nội bộ, localhost, 127.0.0.1; sao chép ca.crt vào thư mục CAs/ của mỗi site để hai site tin nhau khi replicate qua HTTPS.
- docker-compose.tls.yml: override thêm --certs-dir cho hai MinIO; đặt MLFLOW_S3_ENDPOINT_URL=https://... và AWS_CA_BUNDLE cho mlflow-a/b; mount CA cho backup-cron và container mc.
- scripts/lib.sh tự phát hiện TLS đã bật (thử HTTP tới cả hai site) để setup.sh dùng https cho alias và endpoint replication.

Bật bằng: `bash scripts/gen_certs.sh && docker compose -f docker-compose.yml -f docker-compose.tls.yml up -d`, sau đó tin CA trên máy quản trị và cập nhật endpoint replication bằng `mc admin replicate update`. Production dùng chứng chỉ từ CA thật hoặc cert-manager, không dùng `--insecure`.

CHƯƠNG 7. QUY TRÌNH KHÔI PHỤC DỮ LIỆU

Chương này trình bày runbook cho từng kịch bản sự cố. Mỗi runbook gồm: dấu hiệu nhận biết, RPO/RTO kỳ vọng, các bước thực hiện, cách kiểm chứng và lưu ý. Mọi lệnh chạy từ thư mục dự án trong Git Bash sau khi nạp biến môi trường (set -a; source .env; set +a; export MSYS_NO_PATHCONV=1).

7.1. Kịch bản 1 – Xóa nhầm hoặc ghi đè nhầm object

Dấu hiệu: người dùng báo mất file; mc ls không thấy object; hoặc nội dung file bị sai sau lần ingest gần nhất.

RPO / RTO: 0 (mọi version còn nguyên) / ≤ 5 phút.

1. Xem lịch sử version: `mc ls --versions siteA/raw-data/weather/hanoi/2025/2025-12.csv`
2. Nếu bị xóa (có dòng DEL – delete marker): `mc undo siteA/raw-data/weather/hanoi/2025/2025-12.csv` – xóa delete marker, version trước đó trở thành hiện tại.
3. Nếu bị ghi đè sai: `mc cp --version-id <id> siteA/raw-data/.../2025-12.csv siteA/raw-data/.../2025-12.csv` – sao chép version đúng thành version mới nhất.
4. Kiểm chứng ở site B (replication lan cả thao tác khôi phục): `mc ls siteB/raw-data/weather/hanoi/2025/`

Lưu ý: lệnh xóa cũng được replicate sang B, nên replication không thay được backup; chính versioning mới cứu được. Version cũ của raw-data chỉ giữ 14 ngày (ILM) – phát hiện muộn hơn sẽ không khôi phục được.

7.2. Kịch bản 2 – Hỏng hoặc mất dữ liệu trên drive

Dấu hiệu: mc admin info siteA báo drive offline, hoặc ứng dụng vẫn chạy nhưng healing log tăng.

RPO / RTO: 0 / 0 với ≤ 2 drive (đọc tiếp được), vài phút để heal.

1. Xác nhận trạng thái: `mc admin info siteA` (4 drives online, EC:2).
2. Thay drive hỏng (trong demo: xóa nội dung volume để mô phỏng) và khởi động lại MinIO nếu cần.
3. Chạy heal chủ động: `mc admin heal -r siteA/raw-data` (hoặc toàn bộ: `mc admin heal -r siteA`).
4. Kiểm chứng: đọc object bị ảnh hưởng `mc cat siteA/raw-data/.../2025-11.csv | head`; kiểm tra shard đã có lại trên drive.

Lưu ý: mất 3 drive trở lên trên một site thì site đó mất dữ liệu; khi đó chuyển sang kịch bản 4 và đồng bộ lại từ site B bằng mc admin replicate resync.

7.3. Kịch bản 3 – Khôi phục metadata MLflow (không mất site)

Dấu hiệu: MLflow UI mất run/model; postgres-a bị hỏng dữ liệu; hoặc cần quay về trạng thái trước một thay đổi sai.

RPO / RTO: ≤ 5 phút / ≤ 15 phút.

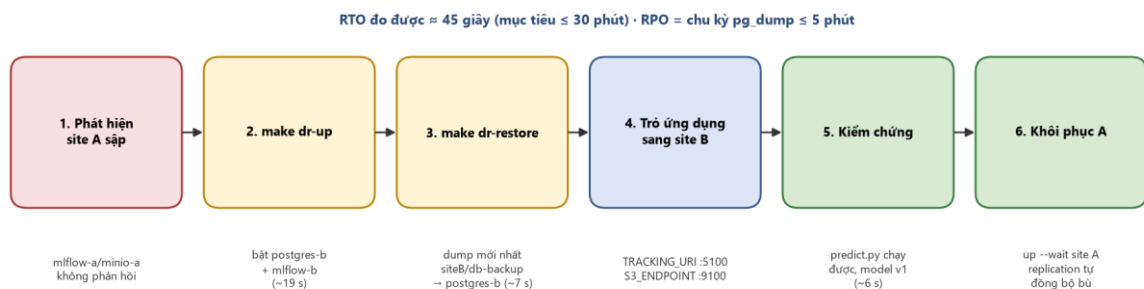
1. Chọn bản dump: `mc ls siteA/db-backup | tail` – tên file chứa thời điểm; chọn bản ngay trước sự cố.
2. Dừng mlflow-a để tránh ghi xen kẽ: `docker compose stop mlflow-a`
3. Restore: `mc cat siteA/db-backup/<file>.sql.gz | docker compose exec -T postgres-a sh -c 'gunzip | psql -q -U mlflow -d mlflow -v ON_ERROR_STOP=1'`
4. Khởi động lại: `docker compose up -d --wait mlflow-a`; mở `http://localhost:5000` kiểm tra run và registry.

Lưu ý: dump có `--clean --if-exists` nên ghi đè toàn bộ database; các run tạo sau thời điểm dump sẽ mất (đúng bằng RPO). Artifact trên MinIO không bị ảnh hưởng vì registry chỉ trỏ tới đường dẫn S3.

7.4. Kịch bản 4 – Mất toàn bộ site A (failover sang site B)

Dấu hiệu: mlflow-a, minio-a không phản hồi; backup-cron báo could not translate host name "postgres-a"; make status lỗi ở siteA.

RPO / RTO: ≤ 5 phút (metadata), vài giây (object) / ≤ 30 phút; đo thực tế 45 giây.



Hình 7.1. Quy trình failover sang site B và khôi phục site A

1. **Xác nhận sự cố** và quyết định failover (tránh failover vì lỗi tạm thời).
2. **Bật hạ tầng dự phòng:** `make dr-up` – khởi động postgres-b và mlflow-b, chờ healthy (~19 s).
3. **Khôi phục metadata:** `make dr-restore` – script `restore_db.sh` lấy bản dump mới nhất trong siteB/db-backup (đã replicate), nạp vào postgres-b, khởi động lại

mlflow-b (~7 s). Có thể chỉ định bản cụ thể: `bash scripts/restore_db.sh mlflow_20260930_012000.sql.gz`

4. **Trở ứng dụng sang site B:** đặt `MLFLOW_TRACKING_URI=http://localhost:5100` và `MLFLOW_S3_ENDPOINT_URL=http://localhost:9100` (trong `.env` hoặc biến môi trường). Credential không đổi vì IAM đã được replicate.
5. **Kiểm chứng:** `python app/predict.py` chạy được, đúng model version; mở `http://localhost:5100` thấy đủ run.
6. **Vận hành trên site B:** `ingest/train/predict` tiếp tục bình thường; object ghi vào minio-b sẽ được replicate ngược về A khi A hoạt động trở lại. Lưu ý backup-cron vẫn trở postgres-a nên tạm thời không có backup mới của postgres-b.

Khôi phục site A (failback)

1. Khởi động lại site A và chờ healthy: `docker compose up -d --wait postgres-a minio-a mlflow-a` (dùng `up --wait`, không dùng `start`, để MLflow thực sự sẵn sàng).
2. Kiểm tra replication tự đồng bộ bù: `make status` – hàng đợi Queued về 0; object ghi ở B trong lúc A sập đã xuất hiện ở A (đo thực tế ~40 s).
3. Nếu đã có run/model mới trên postgres-b trong lúc failover: dump postgres-b và nạp vào postgres-a theo cách của kịch bản 3 (đổi tên container); bước này là thủ công.
4. Trở ứng dụng về site A; tắt hạ tầng dự phòng: `docker compose --profile dr stop mlflow-b postgres-b`

7.5. Kịch bản 5 – Ransomware / tài khoản quản trị bị lộ

Dấu hiệu: audit log có hàng loạt DeleteObject/PutObject bất thường; object bị ghi đè nội dung mã hóa; bản backup bị cố xóa (statusCode 403 hàng loạt trên db-backup).

1. Cô lập: đổi mật khẩu root hai site, vô hiệu hóa user nghi ngờ (mc admin user disable), chặn mạng nếu cần.
2. Đánh giá phạm vi bằng audit log: `docker compose exec audit-log grep "accessKey":"<user>" /logs/audit-site-a.jsonl`.
3. Khôi phục object bị ghi đè/xóa theo kịch bản 1 (version cũ vẫn còn trong 14 ngày với raw-data; mlflow-artifacts không giới hạn).
4. Khôi phục metadata từ db-backup theo kịch bản 3 – bản backup không thể bị xóa trong 24 giờ nhờ COMPLIANCE, kể cả bằng root.
5. Xoay toàn bộ bí mật: sinh `.env` mới cho user ứng dụng, cập nhật user trên MinIO (mc admin user add với mật khẩu mới), khởi động lại dịch vụ.

7.6. Kiểm tra định kỳ

Quy trình khôi phục chỉ có giá trị khi được diễn tập. Nhóm đề xuất lịch: hằng ngày chạy make status kiểm tra replication in sync và có bản backup mới trong 10 phút gần nhất; hằng tuần diễn tập kịch bản 1 và 3; hằng tháng diễn tập kịch bản 4 đầy đủ và ghi lại RTO thực tế. Một sự cố thực tế trong quá trình làm đề tài (mục 8.5) cho thấy backup ngừng hơn 6 giờ mà không ai phát hiện – minh chứng cho việc cần giám sát tự động thay vì chỉ dựa vào cron.

CHƯƠNG 8. TRIỂN KHAI VÀ KIỂM THỬ

8.1. Môi trường

Thành phần	Phiên bản / cấu hình
Máy chủ	Laptop Windows 11, Docker Desktop (WSL2), Git Bash, GNU make, Python 3.12
MinIO	RELEASE.2025-04-22T22-12-26Z; mc RELEASE.2025-04-16
PostgreSQL	16.15-alpine
MLflow	3.16.1 (server và client cùng phiên bản)
Thư viện Python	scikit-learn 1.5+, pandas 2.2+, boto3 1.35+, requests, python-dotenv
Giới hạn RAM	MinIO 1 GB × 2, MLflow 1 GB × 2, Postgres 512 MB × 2, backup-cron 128 MB, audit-log 64 MB

8.2. Trình tự triển khai từ đầu

```
make env      # sinh .env
make up       # 6 container healthy sau ~19 s
make setup    # replication, bucket, lock, SSE, ILM, IAM - 10 s
make ingest   # 3 653 ngày, 120 lô - 5 s
make train    # RandomForest - 10 s
make predict
```

Toàn bộ chuỗi từ trạng thái trống đến có model dự báo mất khoảng 1 phút 30 giây. Kết quả make setup lần đầu (cụm trống): Buckets 4/4, Policies 9/9, Users 4/4, ILM Expiry Rules 3/3 in sync; cả 8 bucket trên hai site đều versioning=Enabled, sse=AES256, db-backup lock=COMPLIANCE.

8.3. Kết quả huấn luyện (tải công việc thực tế)

Chỉ số	Giá trị
Dữ liệu	3 653 ngày (2016-01-01 → 2025-12-31), nguồn open-meteo-archive
Chia tập	Theo thời gian, không xáo trộn: train 2 916 ngày, test 730 ngày (2024–2025)
Mô hình	RandomForestRegressor: 200 cây, max_depth 12, min_samples_leaf 3, 11 đặc trưng
MAE / RMSE / R ²	1,042 °C / 1,435 °C / 0,912
Baseline (ngày mai = hôm nay)	MAE 1,102 °C → mô hình tốt hơn baseline 5,4 %
Dự báo 2026-01-01	19,9 °C

8.4. Kiểm thử các kịch bản sự cố

Kịch bản	Thao tác	Kết quả	Thời gian
Xóa nhầm object	mc rm 2025-12.csv → B mất theo → mc undo	v1 PUT còn nguyên, v2 là delete marker; undo xong B có lại	< 10 s

Kịch bản	Thao tác	Kết quả	Thời gian
Mất 2/4 drive	rm xl.meta của 2025-11.csv trên data1, data2	mc cat vẫn đọc được; heal Green 100 %; 4 drive có lại shard	~30 s
Least privilege	mlflow-app ghi raw-data; raw-reader ghi; mlflow-app gọi admin	Insufficient permissions / Access Denied; 45 sự kiện 403 trong audit log	—
Mã hóa trên đĩa	grep temp_mean trên volume minio-a-data1	0 file chứa plaintext; mc cat qua API vẫn đọc được	—
Object Lock	mc retention info; mc stat bản backup	COMPLIANCE 1DAYS; Retain-Until-Date = +24 h	—
Backup định kỳ	Quan sát db-backup 3 chu kỳ	Bản mới mỗi 5 phút (01:00, 01:05...), xuất hiện ở B sau vài giây	—
ILM db-backup	mc ilm rule ls hai site	backup-expire-7d + backup-clean-delmarker trên cả A và B	—
Failover mất site A	stop 3 container A → dr-up → dr-restore → predict qua :5100	Registry B khớp A (model v1); predict 19,9 °C; ghi vào B OK	45 s
Failback	up --wait site A → status → stop site B	File ghi ở B lúc A sập tự sang A sau ~43 s; Queued 0; backup chạy lại	~55 s

Đáng chú ý: RTO đo được cho kịch bản mất site A là 45 giây, thấp hơn mục tiêu 30 phút 40 lần, nhờ hạ tầng dự phòng được định nghĩa sẵn trong compose và toàn bộ quy trình gói trong hai lệnh make. RPO thực tế bằng khoảng cách từ bản dump cuối tới thời điểm sự cố, tối đa 5 phút.

8.5. Sự cố gặp phải và bài học

- **Backup ngừng hoạt động hơn 6 giờ do CRLF:** file backup_db.sh bị lưu với xuống dòng Windows (CRLF); bash trong container Linux báo ": invalid option name" và không tạo bản backup nào. Phát hiện chỉ nhờ so sánh thời điểm bản backup mới nhất với hiện tại. Khắc phục: chuyển sang LF và thêm .gitattributes (*.sh text eol=lf). Bài học: cần giám sát tuổi của bản backup mới nhất, không tin cron chạy là đủ.
- **Endpoint replication:** dùng localhost:9100 làm endpoint site B thất bại vì bên trong container minio-a localhost là chính nó; phải dùng tên nội bộ qua network replication.
- **ILM không được replicate đầy đủ:** rule ExpiredObjectDeleteMarker không sang site B; xử lý bằng cách nhập rule ILM cho db-backup lên cả hai site.
- **Trạng thái in sync báo sớm:** ngay sau khi nhập ILM, mc admin replicate status báo 1/3 in sync trong vài giây; setup.sh được bổ sung vòng chờ ILM để tránh báo động giả.

- **MSYS_NO_PATHCONV trên Git Bash:** thiếu biến này, Git Bash đổi /d/... thành D:/... khiến docker run -v ... rm chạy vào đường dẫn không tồn tại mà không báo lỗi; mọi script đã export biến này.
- **Phát hiện TLS khi site A sập:** phiên bản đầu của lib.sh chỉ thử HTTP tới site A nên khi A sập tương TLS đã bật; sửa thành thử cả hai site.

CHƯƠNG 9. ĐÁNH GIÁ, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

9.1. Đánh giá theo yêu cầu đề bài

Yêu cầu	Đáp ứng	Bằng chứng
Kiến trúc đề xuất	Hai site active-active ở mức object, warm standby ở mức DB/MLflow; 3 network tách biệt	Chương 3, Hình 3.1, docker-compose.yml
Loại dữ liệu cần lưu trữ	6 nhóm dữ liệu, phân loại 4 mức quan trọng, mỗi nhóm có cơ chế bảo vệ riêng	Chương 4
Sao lưu/khôi phục đáp ứng RPO	RPO vài giây (object), ≤ 5 phút (metadata); 4 lớp phòng thủ; bản backup bất biến	Chương 5, bảng 5.6
Bảo mật dữ liệu	8 lớp: bí mật, IAM, SSE-S3, WORM, versioning, audit, mạng, TLS	Chương 6, kiểm chứng 8.4
Quy trình khôi phục	5 runbook + lịch diễn tập; failover đo được 45 s	Chương 7, 8.4

9.2. Ưu điểm

- Toàn bộ mã nguồn mở, không phụ thuộc nhà cung cấp cloud; tái lập được trên máy cá nhân bằng một chuỗi lệnh.
- Site Replication đồng bộ cả IAM và cấu hình nên failover không cần cấu hình lại credential hay bucket.
- Bản sao lưu bất biến (COMPLIANCE) kết hợp user chỉ-ghi tạo tuyến phòng thủ hiệu quả chống ransomware với chi phí gần bằng 0.
- Mọi bước đều idempotent và có kiểm tra tự động (healthcheck, vòng chờ đồng bộ, gzip -t), giảm lỗi do con người.
- Số liệu RPO/RTO không chỉ là mục tiêu trên giấy mà được đo bằng thực nghiệm.

9.3. Hạn chế

- Hai site chạy trên cùng một máy vật lý: không mô phỏng được độ trễ WAN, băng thông hạn chế và lỗi phân vùng mạng thật; 4 drive là 4 volume trên cùng một đĩa nên EC chỉ chứng minh logic chịu lỗi, không bảo vệ khi đĩa vật lý hỏng.
- Metadata MLflow chỉ được bảo vệ bằng dump định kỳ (RPO 5 phút), chưa có streaming replication; sau failover, thay đổi trên postgres-b không tự về postgres-a.
- KMS tính một khóa, không xoay khóa; TLS tắt mặc định.
- Chưa có giám sát và cảnh báo tự động (tuổi backup, hàng đợi replication, drive offline) – sự cố CRLF cho thấy đây là lỗi hồng vận hành thực tế.
- Failover vẫn là thao tác thủ công (2 lệnh make), chưa tự động phát hiện và chuyển đổi.

9.4. Hướng phát triển

1. Triển khai hai site trên hai máy/VM riêng, hoặc hai vùng cloud, để đo RPO thực với độ trễ mạng; dùng MinIO multi-node để EC trải trên nhiều máy.
2. Bổ sung streaming replication PostgreSQL A → B (hot standby) để RPO metadata gần 0, giữ pg_dump định kỳ như lớp chống xóa nhầm.
3. Thay KMS tĩnh bằng KES + HashiCorp Vault; bật TLS mặc định với chứng chỉ từ CA nội bộ hoặc cert-manager.
4. Giám sát bằng Prometheus (MinIO xuất metrics sẵn) + Alertmanager: cảnh báo khi bản backup mới nhất quá 10 phút, Queued replication tăng, drive offline; đẩy audit log vào Loki/Elasticsearch.
5. Tự động hóa failover bằng health probe và DNS/reverse proxy (ví dụ Traefik) chuyển endpoint sang site B; tự động dump-and-restore ngược khi failback.
6. Mở rộng dữ liệu: nhiều trạm quan trắc, dữ liệu theo giờ; áp dụng cùng khung phân loại và RPO/RTO cho từng nhóm dữ liệu mới.

KẾT LUẬN

Đề tài đã xây dựng và kiểm chứng một giải pháp lưu trữ – sao lưu – khôi phục thảm họa đa site hoàn chỉnh cho hệ thống dự báo thời tiết dựa trên học máy. Kiến trúc hai site với MinIO Site Replication bảo vệ dữ liệu object ở mức drive (erasure coding EC 2+2) và mức site; versioning cùng ILM bảo vệ trước thao tác nhầm; sao lưu PostgreSQL định kỳ 5 phút vào bucket bất biến (Object Lock COMPLIANCE) bảo vệ siêu dữ liệu trước cả ransomware. Sáu nhóm dữ liệu được phân loại và gán RPO/RTO riêng, trong đó dữ liệu quan trọng nhất đạt RPO vài giây và RTO đo được 45 giây cho kịch bản mất toàn bộ site chính.

Giải pháp bảo mật tám lớp – quản lý bí mật, phân quyền tối thiểu, mã hóa lưu trữ, bất biến, versioning, audit log, phân vùng mạng và TLS – được kiểm chứng bằng thực nghiệm chứ không chỉ mô tả. Năm runbook khôi phục bao phủ các kịch bản từ xóa nhầm một file đến mất nguyên site, và những sự cố gặp phải trong quá trình thực hiện (backup ngừng do CRLF, endpoint replication sai, ILM không đồng bộ đầy đủ) đã trở thành bài học được đưa thẳng vào thiết kế và quy trình.

Hạn chế chính nằm ở môi trường mô phỏng trên một máy và việc thiếu giám sát tự động; đây cũng là các hướng phát triển tiếp theo. Toàn bộ mã nguồn, cấu hình và tài liệu vận hành được tổ chức để bất kỳ ai cũng có thể tái lập hệ thống trong khoảng 90 giây và tự diễn tập lại các kịch bản khôi phục.

TÀI LIỆU THAM KHẢO

1. MinIO, Inc. MinIO Object Storage Documentation: Erasure Coding, Site Replication, Object Locking, Server-Side Encryption, Bucket Lifecycle Management. <https://min.io/docs/minio/linux/> (truy cập 9/2026).
2. MinIO, Inc. MinIO Client (mc) Reference. <https://min.io/docs/minio/linux/reference/minio-mc.html>.
3. The PostgreSQL Global Development Group. PostgreSQL 16 Documentation, Chapter 26 – Backup and Restore. <https://www.postgresql.org/docs/16/backup.html>.
4. MLflow Project. MLflow 3.x Documentation: Tracking Server, Model Registry, Artifact Stores. <https://mlflow.org/docs/latest/>.
5. Docker, Inc. Docker Compose Specification: services, healthcheck, depends_on, profiles, networks. <https://docs.docker.com/compose/>.
6. Amazon Web Services. Amazon S3 API Reference; S3 Object Lock; Bucket policies and IAM policy language. <https://docs.aws.amazon.com/s3/>.
7. Open-Meteo. Historical Weather API. <https://open-meteo.com/en/docs/historical-weather-api>.
8. Plank, J. S. (1997). A tutorial on Reed-Solomon coding for fault-tolerance in RAID-like systems. *Software: Practice and Experience*, 27(9), 995–1012.
9. Preston, W. C. (2021). *Modern Data Protection: Ensuring Recoverability of All Modern Workloads*. O'Reilly Media.
10. NIST. SP 800-34 Rev. 1: Contingency Planning Guide for Federal Information Systems (định nghĩa RPO/RTO).

PHỤ LỤC A. BÀN PHÂN CÔNG CÔNG VIỆC CỦA NHÓM

Nhóm 13 gồm ... thành viên. Công việc được chia theo các mảng của đề tài; mỗi mảng có một người phụ trách chính và một người hỗ trợ/kiểm tra chéo. Bảng dưới liệt kê nhiệm vụ, sản phẩm cụ thể và mức độ hoàn thành (nhóm điền tên thành viên vào cột tương ứng).

STT	Nhiệm vụ	Sản phẩm / file	Phụ trách	Hỗ trợ	Hoàn thành
1	Khảo sát yêu cầu, phân loại dữ liệu, xác định RPO/RTO	Chương 1, 4; bảng 4.2, 5.6			100 %
2	Nghiên cứu cơ sở lý thuyết MinIO/EC/replication/WORM	Chương 2			100 %
3	Thiết kế kiến trúc, mạng, compose	Chương 3; docker-compose.yml, docker-compose.tls.yml, docs/architecture.md			100 %
4	Script thiết lập lưu trữ (replication, bucket, lock, SSE, ILM, IAM)	scripts/setup.sh, lib.sh, policies/*.json			100 %
5	Sao lưu PostgreSQL định kỳ, restore, failover	docker/backup-cron/*, scripts/backup_db.sh, restore_db.sh, Makefile			100 %
6	Bảo mật: bí mật, TLS, audit log	scripts/gen_env.sh, gen_certs.sh, docker/audit-log/receiver.py, docs/security.md			100 %
7	Ứng dụng ML: ingest, train, predict	app/common.py, ingest.py, train.py, predict.py			100 %
8	Kiểm thử các kịch bản, đo RPO/RTO, ghi nhận sự cố	Chương 8; docs/demo.md			100 %
9	Viết và biên tập báo cáo, vẽ sơ đồ	Báo cáo này			100 %
10	Chuẩn bị và trình bày demo	docs/demo.md, slide			100 %

Nhật ký làm việc nhóm (tóm tắt)

Tuần	Nội dung	Kết quả
1	Họp chọn đề tài, phân tích yêu cầu, thống nhất công nghệ (MinIO, PostgreSQL, MLflow, Docker)	Đề cương, bảng phân loại dữ liệu sơ bộ
2	Dựng compose hai site, thiết lập Site Replication, xử lý lỗi endpoint localhost	make up / make setup chạy được
3	IAM, SSE-S3, Object Lock, ILM; viết ứng dụng ingest/train/predict	Luồng ML chạy end-to-end, model v1
4	Backup cron, restore, failover; phát hiện và sửa sự cố CRLF; bổ sung ILM db-backup	make dr-up / dr-restore; RTO 45 s

Tuần	Nội dung	Kết quả
5	Kiểm thử toàn bộ kịch bản, dựng lại từ đầu để xác nhận, viết báo cáo và kịch bản demo	Báo cáo, docs/demo.md

PHỤ LỤC B. MÃ NGUỒN VÀ CẤU HÌNH CHÍNH

B.1. Cấu trúc thư mục dự án

```

LTDL-N13/
├── Makefile                # lỗi tắt vận hành
├── docker-compose.yml      # 2 site, 3 network, 11 volume
├── docker-compose.tls.yml  # override bật TLS
├── .env.example / .env     # bí mật (sinh bằng make env)
├── app/ common.py ingest.py train.py predict.py requirements.txt
├── docker/ mlflow/Dockerfile backup-cron/{Dockerfile,entrypoint.sh} audit-
log/receiver.py
├── policies/ mlflow-artifacts-rw.json raw-data-readonly.json raw-data-ingest.json
db-backup-writer.json
├── scripts/ lib.sh setup.sh backup_db.sh restore_db.sh status.sh gen_env.sh
gen_certs.sh
├── certs/                  # sinh bằng make certs (gitignore)
└── docs/ architecture.md security.md demo.md

```

B.2. Trích docker-compose.yml – dịch vụ minio-a và backup-cron

```

x-minio: &minio
  image: quay.io/minio/minio:RELEASE.2025-04-22T22-12-26Z
  command: server /data{1...4} --console-address ":9001"
  healthcheck: { test: ["CMD", "mc", "ready", "local"], interval: 5s, retries: 20 }
  services:
    minio-a:
      <<: *minio
      environment:
        MINIO_ROOT_USER: ${MINIO_A_ROOT_USER}
        MINIO_ROOT_PASSWORD: ${MINIO_A_ROOT_PASSWORD}
        MINIO_SITE_NAME: site-a
        MINIO_KMS_SECRET_KEY: ${MINIO_KMS_SECRET_KEY}
        MINIO_AUDIT_WEBHOOK_ENABLE_audit: "on"
        MINIO_AUDIT_WEBHOOK_ENDPOINT_audit: http://audit-log:8080/site-a
      ports: ["9000:9000", "9001:9001"]
      volumes: [minio-a-data1:/data1, minio-a-data2:/data2, minio-a-data3:/data3,
minio-a-data4:/data4]
      networks: [site-a, replication]
    backup-cron:
      build: { context: ., dockerfile: docker/backup-cron/Dockerfile }
      environment:
        BACKUP_SCHEDULE: "*/5 * * * *"
        PGHOST: postgres-a
        MC_HOST_siteA: http://${BACKUP_ACCESS_KEY}:${BACKUP_SECRET_KEY}@minio-a:9000
      volumes: [./scripts/backup_db.sh:/scripts/backup_db.sh:ro]
      depends_on: { postgres-a: { condition: service_healthy }, minio-a: { condition:
service_healthy } }

```

B.3. Trích scripts/setup.sh – bucket, Object Lock, IAM

```

for b in raw-data mlflow-artifacts logs; do
  mc mb --ignore-existing "siteA/$b"; mc version enable "siteA/$b"
done
mc mb --with-lock siteA/db-backup # --with-lock tự bật
versioning
mc retention set --default COMPLIANCE 1d siteA/db-backup
for b in "${BUCKETS[@]}"; do mc encrypt set sse-s3 "siteA/$b"; done

ensure_user() { # <access> <secret> <policy-name> <policy-file>
  mc admin policy create siteA "$3" "$4"
}

```

```
mc admin user add siteA "$1" "$2"
mc admin policy attach siteA "$3" --user "$1"
}
ensure_user "$MLFLOW_APP_ACCESS_KEY" "$MLFLOW_APP_SECRET_KEY" mlflow-artifacts-rw
policies/mlflow-artifacts-rw.json
ensure_user "$RAW_READER_ACCESS_KEY" "$RAW_READER_SECRET_KEY" raw-data-readonly
policies/raw-data-readonly.json
ensure_user "$INGEST_ACCESS_KEY" "$INGEST_SECRET_KEY" raw-data-ingest
policies/raw-data-ingest.json
ensure_user "$BACKUP_ACCESS_KEY" "$BACKUP_SECRET_KEY" db-backup-writer
policies/db-backup-writer.json
```

B.4. Policy db-backup-writer.json

```
{ "Version": "2012-10-17", "Statement": [
  { "Effect": "Allow", "Action": ["s3:GetBucketLocation", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::db-backup"] },
  { "Effect": "Allow", "Action": ["s3:PutObject", "s3:GetObject"],
    "Resource": ["arn:aws:s3:::db-backup/*"] } ] }
```

B.5. scripts/restore_db.sh

```
name="${1:-$(mc ls siteB/db-backup | awk '{print $NF}' | grep '\.sql\.gz$' | sort |
tail -1)}"
compose exec -T postgres-b pg_isready -U mlflow -d mlflow || die "postgres-b chưa
chạy – make dr-up trước"
mc cat "siteB/db-backup/$name" \
| compose exec -T postgres-b sh -c 'gunzip | psql -q -U mlflow -d mlflow -v
ON_ERROR_STOP=1'
compose restart mlflow-b
```

B.6. Trích app/common.py – client S3 dùng user ứng dụng

```
def s3_client(access_env, secret_env):
    return boto3.client("s3", endpoint_url=S3_ENDPOINT,
        aws_access_key_id=os.environ[access_env],
        aws_secret_access_key=os.environ[secret_env],
        region_name="us-east-1",
        config=Config(signature_version="s3v4", s3={"addressing_style": "path"}))

# ingest.py: s3_client("INGEST_ACCESS_KEY", "INGEST_SECRET_KEY")
# train.py / predict.py: s3_client("RAW_READER_ACCESS_KEY",
"RAW_READER_SECRET_KEY")
# init_mlflow(): AWS_ACCESS_KEY_ID = MLFLOW_APP_ACCESS_KEY (artifact)
```